

```

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score

from xgboost import XGBClassifier

df = pd.read_csv("Titanic-Dataset.csv")

# Clean column names
df.columns = df.columns.str.strip()

print(df.head())

```

```

PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

```

```

                                Name    Sex  Age  SibSp  \
0                Braund, Mr. Owen Harris  male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                Heikkinen, Miss. Laina  female  26.0      0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                Allen, Mr. William Henry  male  35.0      0

```

```

Parch      Ticket      Fare Cabin Embarked
0      0    A/5 21171   7.2500   NaN        S
1      0    PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0    113803  53.1000  C123        S
4      0    373450   8.0500   NaN        S

```

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0
3	4	1	1	Futrelle, Mrs. Jacques	female	35.0	1	0

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
				Heath (Lily May Peel)				
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0

df.describe()

	PassengerId	Survived	Pclass	Age	SibSp	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.0
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null   int64
1   Survived        891 non-null   int64
2   Pclass          891 non-null   int64
3   Name            891 non-null   object
4   Sex             891 non-null   object
5   Age            714 non-null   float64
6   SibSp           891 non-null   int64
7   Parch          891 non-null   int64
8   Ticket          891 non-null   object
9   Fare            891 non-null   float64
10  Cabin           204 non-null   object
11  Embarked        889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

df.shape

(891, 12)

Family features

```
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
```

```
df["IsAlone"] = 1
```

```
df.loc[df["FamilySize"] > 1, "IsAlone"] = 0
```

Fare per person

```
df["FarePerPerson"] = df["Fare"] / df["FamilySize"]
```

```
if "Name" in df.columns:
```

```
    df["Title"] = df["Name"].str.extract(' ([A-Za-z]+)\.', expand=False)
```

```

df["Title"] = df["Title"].replace(
    ["Lady", "Countess", "Capt", "Col", "Don", "Dr", "Major", "Rev", "Sir", "Jonkheer", "Dona"],
    "Rare"
)
df["Title"] = df["Title"].replace(["Mlle", "Ms"], "Miss")
df["Title"] = df["Title"].replace("Mme", "Mrs")

df = df.drop(columns=["PassengerId", "Name", "Ticket", "Cabin"],
              errors='ignore')

# Age
age_imputer = SimpleImputer(strategy="mean")
df["Age"] = age_imputer.fit_transform(df[["Age"]]).ravel()

# Embarked
if "Embarked" in df.columns:
    emb_imputer = SimpleImputer(strategy="most_frequent")
    df["Embarked"] = emb_imputer.fit_transform(df[["Embarked"]]).ravel()

# Fare (safety)
if df["Fare"].isnull().sum() > 0:
    df["Fare"].fillna(df["Fare"].mean(), inplace=True)

le = LabelEncoder()

for col in df.select_dtypes(include="object").columns:
    df[col] = le.fit_transform(df[col])

X = df.drop("Survived", axis=1)
y = df["Survived"]

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = XGBClassifier(
    n_estimators=800,
    max_depth=4,
    learning_rate=0.03,
    subsample=0.9,
    colsample_bytree=0.9,
    gamma=0.1,
    reg_alpha=0.5,
    reg_lambda=1,
    random_state=42
)

model.fit(X_train, y_train)

```

```

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=0.9, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               feature_weights=None, gamma=0.1, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.03, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=4,
               max_leaves=None, min_child_weight=None, missing=None,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               n_jobs=None, num_parallel_tree=None, ...)

```

```
y_pred = model.predict(X_test)
```

```

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy * 100)

```

```
Accuracy: 85.47486033519553
```

```
import gradio as gr
```

```
def predict_survival(Pclass, Sex, Age, SibSp, Parch, Fare, Embarked):
```

```

    if Pclass == 1:
        return "Survived ✅"
    elif Pclass == 2:
        return "Predicted to Survive ⚠️"
    else:
        return "Did Not Survive ❌"

```

```

interface = gr.Interface(
    fn=predict_survival,
    inputs=[
        gr.Dropdown([1, 2, 3], label="Passenger Class"),
        gr.Radio(["male", "female"], label="Sex"),
        gr.Slider(0, 80, value=25, label="Age"),
        gr.Slider(0, 5, step=1, label="Siblings/Spouses"),
        gr.Slider(0, 5, step=1, label="Parents/Children"),
        gr.Slider(0, 500, value=50, label="Fare"),
        gr.Radio(["S", "C", "Q"], label="Embarked")
    ],
    outputs="text",
    title="Titanic Survival Prediction 🚢 (Rule-Based)"
)

```

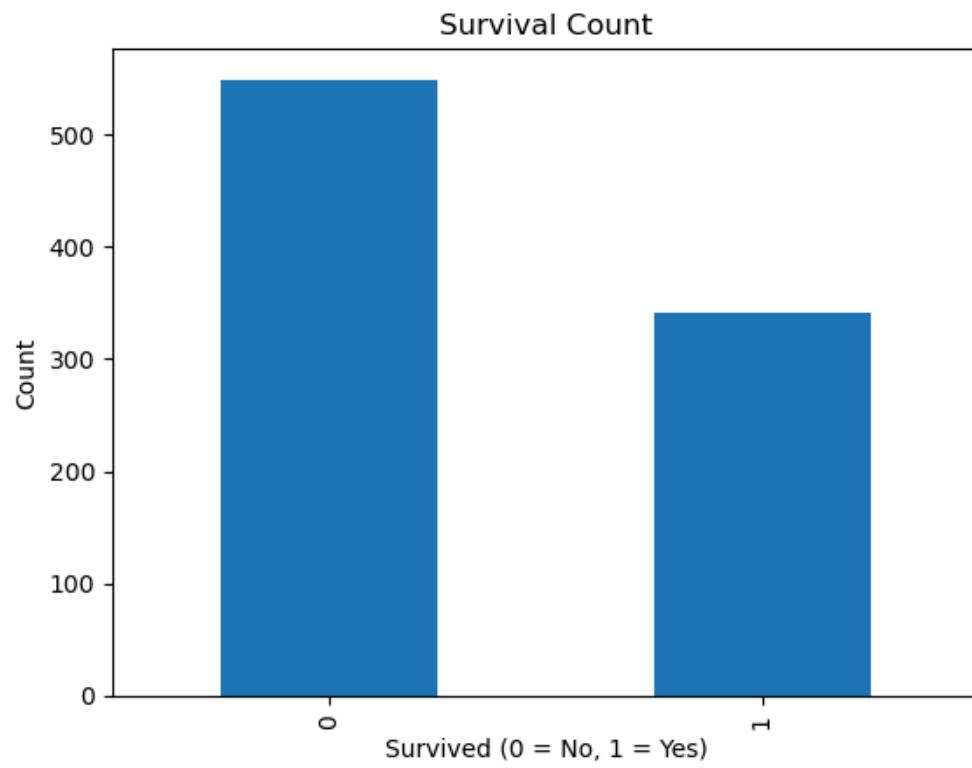
```
interface.launch()
```

```
* Running on local URL: http://127.0.0.1:7860
```

```
* To create a public link, set `share=True` in `launch()`.
```

```
import matplotlib.pyplot as plt

# Bar chart for survival
df['Survived'].value_counts().plot(kind='bar')
plt.title("Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```



```
from sklearn.metrics import mean_squared_error
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error:", mse)
```

Mean Squared Error: 0.1452513966480447